North Bengal International University

Lab Report: 01

Exp Name : Linear search & Binary Search Algorithms

Subject : Analysis of algorithm Sessional

Subject Code : CSE 3111

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE(26)
Roll : 0712340012101005
Semester : 3rd Year, 5th semester.

Submitted To:

Name : Afsana Tasnim
Designation : (HOD)Lecturer
Dept : CSE
Date :

Task No: 01

Task Name: Linear search

Introduction: Linear Search is the simplest searching algorithm used to find a particular element in a list or array. It works by checking each element one by one from the beginning until the desired value is found or the list ends

Algorithm:

- Start from the first element of the array.
- Compare the current element with the target value.
- If it matches, return the index (or position).
- If it doesn't match, move to the next element.
- Repeat steps 2–4 until the end of the array.
- If the target is not found, return a Not found

Pseudocode:

```
START
   READ n
   READ array[0..n-1]
   READ key

   FOR i = 0 TO n-1
      IF array[i] == key THEN
         PRINT "Found at position", i+1
         STOP
      ENDIF
   NEXT i

   PRINT "Not found"
END
```

code:

```c
#include <stdio.h>

int main() {
    int n, i, key, found = 0;

    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        printf("Element %d: ", i);
        scanf("%d", &arr[i] );
    }

    printf("\n Enter the number to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++) {
        if(arr[i] == key) {
            printf("Number %d found at index %d (position %d).\n", key, i, i+1);
            found = 1;
            break;
        }
    }

    if(!found) {
        printf("Number %d not found in the array.\n", key);
    }

    return 0;
}
```

output:

OUTPUT TERMINAL

```
Enter the number of elements in the array: 5
Enter 5 elements:
Element 0: 1 2 3 4 5
Element 1: Element 2: Element 3: Element 4:
 Enter the number to search: 5
Number 5 found at index 4 (position 5).
```

Task No: 02

Task Name: Binary search

Introduction: Binary search is a fast way to find a value in a **sorted** array (ascending order). It repeatedly halves the search range by comparing the middle element with the target.

Algorithm:

- Read n (size of array) and the n elements **in ascending order**.
- Read the target value x.
- Set low = 0, high = n - 1.
- While low <= high:
  - Compute mid = low + (high - low) / 2.
  - If A[mid] == x: **found** → stop.
  - Else if A[mid] < x: search right half → low = mid + 1.
  - Else (A[mid] > x): search left half → high = mid - 1.

If the loop finishes without finding, x is **not found**.

Pseudocode:

```
low  ← 0
high ← n - 1
foundIndex ← -1

WHILE low ≤ high DO
   mid ← low + (high - low) / 2
   IF A[mid] = x THEN
      foundIndex ← mid
      BREAK
   ELSE IF A[mid] < x THEN
      low ← mid + 1
   ELSE
      high ← mid - 1
   END IF
END WHILE

IF foundIndex = -1 THEN
   OUTPUT "Not found"
ELSE
   OUTPUT "Found at index " + foundIndex   // 0-based index
END IF
```

Code:

```c
#include <stdio.h>

int main() {
    int arr[100], n, target;
    int low, high, mid;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d sorted elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter value to search: ");
    scanf("%d", &target);

    low = 0;
    high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == target) {
            printf("Found at index %d\n", mid);
            return 0;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    printf("Not found\n");
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 sorted elements:
1 3 5 7 8 9
Enter value to search: 5
Found at index 2
PS C:\Users\Lenovo\Documents\5th semester\Analysis of Algorithm\lab>
```

Conclusion: Conclusion: Linear search checks each element one by one and works on unsorted data, but it's slower for large arrays. Binary search is much faster with a time complexity of O(log n), but it only works on sorted arrays. So, use linear search for small or unsorted data, and binary search when speed matters and the data is sorted.